

The Untracked Key

The three cloud registrations from doc 2 all failed with a confusing 404. Chasing it down turned up a real auth check running in production that had never once been committed to this repo.

gateway: generic-mcp-gateway — image 338d8ba pushed: 338d8ba..1e540a7 2026-08-16

00 The symptom

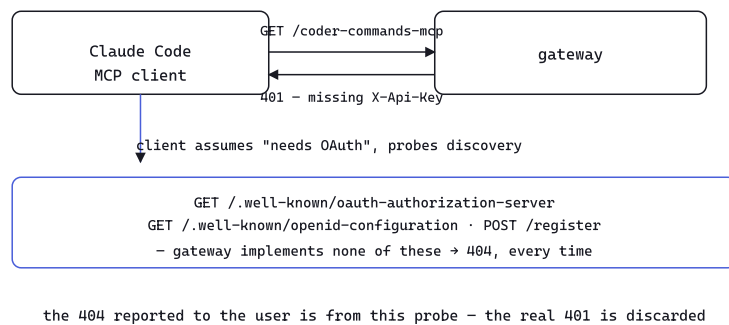
`claude mcp list` failed all three fresh registrations identically:

```
HTTP 404: Invalid OAuth error response: ZodError: [ ... ] Raw
body: {"detail":"Not Found"}
```

That reads like a routing problem — the wrong path, or a missing proxy route. It wasn't. The gateway's own `/healthz` endpoint had already been confirmed reachable from this laptop moments earlier via plain `curl`, so the network path and the gateway process itself were never actually in question.

01 What the 404 actually was

A direct `curl` to the exact same URL returned `401 Unauthorized`, not 404 — a completely different, more informative signal that Claude Code's own error handling had discarded.



Confirmed on the gateway's own `docker logs`: every failed registration left a trail of `404` lines for these exact discovery paths, immediately after the real `401`.

02 Why the 401 was surprising at all

The gateway's proxy code (`api/main.py`) is a pure byte-level relay — method, headers, body, and status code all forwarded as-is between the caller and whichever upstream MCP server the path names. Reading the file as checked out on this laptop showed no auth logic anywhere: no header check, no key comparison, nothing. A 401 from a proxy with no auth code should have been impossible.

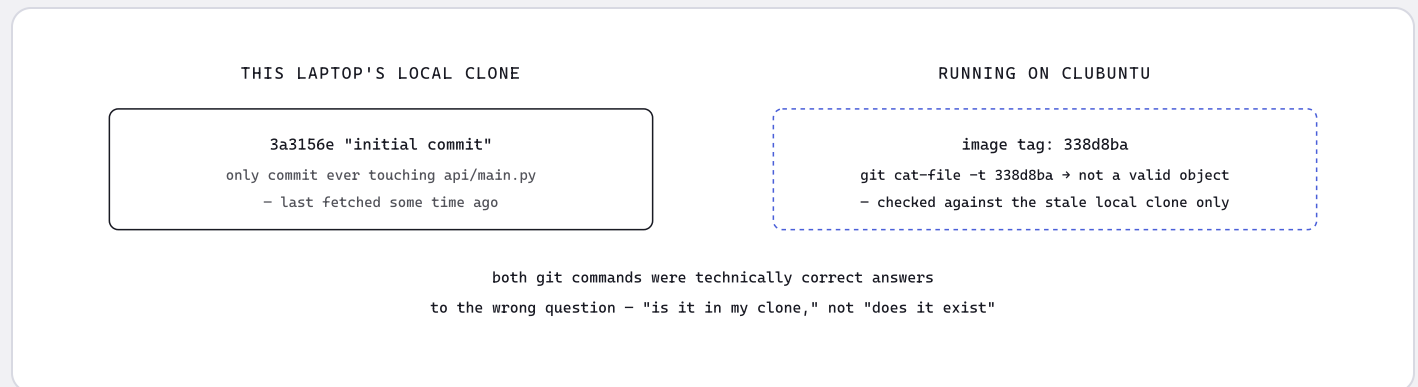
It wasn't impossible — it meant the code actually running in the container wasn't the code in this checkout.

03 The commit that seemed to come from nowhere

`docker exec generic-mcp-gateway cat /app/main.py` pulled the real, running source straight out of the container. It had a full `require_api_key` dependency, checked against a `GATEWAY_API_KEY` environment variable already set on the container — none of which existed in the local file on this laptop. `git log --oneline -- api/main.py`, run against this laptop's own checkout, showed exactly one commit ever touching that file. `git cat-file -t 338d8ba` — the image tag actually deployed — failed outright: `fatal: Not a valid object name`.

At that point it looked like the image had been built from a commit that existed nowhere at all — an image with no history behind it. That reading turned out to be wrong in one

specific, important way: `git cat-file` and `git log` only ever look at *this laptop's local clone*. Nobody had run `git fetch` yet.



Section 09 covers what a `git fetch` actually revealed — the commit was real, on GitHub, the whole time.

04 Immediate fix — authenticate the client

Read the real key straight off the running container (`docker inspect generic-mcp-gateway`, `GATEWAY_API_KEY` env var) and re-registered all three MCP servers with it attached as a header:

```
RE-REGISTRATION, WITH AUTH

claude mcp remove codercommands -s user
claude mcp remove docker -s user
claude mcp remove postgres -s user

claude mcp add --transport http codercommands http://clubuntu.dala-cirius.ts.net:8100/coder-co
-H "X-API-Key: <key>" -s user
claude mcp add --transport http docker http://clubuntu.dala-cirius.ts.net:8100/docker-mcp \
-H "X-API-Key: <key>" -s user
claude mcp add --transport http postgres http://clubuntu.dala-cirius.ts.net:8100/postgres-mcp
-H "X-API-Key: <key>" -s user
```

All three then showed `✓ Connected`. This fixed the symptom — this laptop can now reach the cloud gateway — but left the actual gap untouched: the tracked source still didn't match production.

05 The real fix — reconstructing the source of truth

If `build.yml` ran again from this repo's `main` as it stood, it would have built and pushed a gateway image *without* the auth check — silently replacing the currently-protected gateway with an open one the next time a tag triggered deploy. `docker-mcp` sits behind this same gateway holding the host's Docker socket, so "silently open" here is not a small gap.

Ported the auth code back out of the live container into `api/main.py`, then verified it byte-for-byte against the running version with a plain `diff` before committing — not just "looks equivalent," actually identical.

FILE	CHANGE
<code>api/main.py</code>	restored <code>require_api_key</code> dependency (HMAC-safe comparison against <code>GATEWAY_API_KEY</code>), applied to all three proxy routes; <code>/healthz</code> stays exempt for the compose healthcheck
<code>docker-compose.yml</code>	gateway service now passes <code>GATEWAY_API_KEY: \${GATEWAY_API_KEY}</code> through to the container
<code>docker-compose.deploy.yml</code>	same passthrough, mirrored
<code>.env.example</code>	documents the variable and why it fails closed, with a generation command
<code>.github/workflows/build.yml</code>	throwaway <code>GATEWAY_API_KEY=ci</code> added alongside the other CI placeholder values so <code>docker compose build</code> still parses cleanly with no real <code>.env</code> present

clubuntu's own `.env` already had a real `GATEWAY_API_KEY` set (confirmed by reading it over SSH) — this laptop's tracked compose files just weren't passing it through. Committed locally as `d7dddc`.

06 Pushing surfaced the real answer

`git push origin main` was rejected: `! [rejected] main → main (fetch first)`. Remote had commits this laptop had never fetched. `git fetch origin` pulled them down — and the mystery resolved itself in one line: `338d8ba` was sitting right there on `origin/main`, titled *"Require an API key on the gateway; expose it on the tailnet."*

Not an orphan. Not built outside the pipeline. It had been pushed straight to GitHub's `main` at some point, the self-hosted runner's `build.yml` had picked it up exactly as designed, built it, and pushed the image — the pipeline had worked correctly the entire time. The only thing that had actually gone wrong was that this laptop's own clone had fallen behind and nobody had run `git fetch` since.



real commit, pushed straight to GitHub, built by the normal pipeline -
this laptop's clone was simply behind, not the repo's history

Comparing the two independently-written versions of `api/main.py` — the real commit's and this laptop's reconstruction — showed them byte-identical. The reconstruction had been exactly right.

07 Reconciling the two histories

`338d8ba` turned out to be a strict superset of the local reconstruction — same `api/main.py` byte-for-byte, plus `tsdproxy.enable` / `tsdproxy.ephemeral` labels on the gateway service (wiring it into clubuntu's Tailscale-proxy container) that the reconstruction had no way to know about. The one thing the reconstruction had that `338d8ba` didn't: the `GATEWAY_API_KEY=ci` placeholder in `build.yml`, needed so a fresh CI checkout doesn't warn on an unset variable.

Resolution: `git reset --hard origin/main` to adopt the real commit as the base, discarding the now-redundant local reconstruction, then a small one-line follow-up commit adding just the missing `build.yml` placeholder on top.

RECONCILIATION

```
git reset --hard origin/main
# ...re-add GATEWAY_API_KEY=ci to build.yml...
git commit -m "Add GATEWAY_API_KEY CI placeholder to build.yml"
git push origin main
```

08 Verified

CHECK	RESULT
Reconstructed <code>api/main.py</code> vs. live container's <code>/app/main.py</code>	✓ byte-identical (<code>diff</code> , no output)
Reconstructed <code>api/main.py</code> vs. the real <code>338d8ba</code> commit	✓ also byte-identical — independent confirmation

CHECK	RESULT
<code>docker compose config</code> , post-reconciliation	✓ parses cleanly
Three MCP registrations, re-added with header	✓ all ✓ Connected
Pushed to <code>origin/main</code>	✓ 338d8ba..1e540a7, fast-forward

09 What this leaves open

- ▶ A working production system and a locally-checked-out repository can drift apart silently — not because history was lost, but because `git fetch` hadn't run. Nothing here failed loudly until an unrelated task (registering a new client) happened to probe the gap.
- ▶ Diagnosing the connection failure meant not trusting the first error shown — Claude Code's reported 404 was real but secondary; going one layer down with a plain `curl` surfaced the true 401 immediately.
- ▶ Diagnosing the "orphan commit" meant recognizing that `git log` and `git cat-file` answer "is this in my local clone," never "does this exist" — a `git fetch` should be the reflex the moment a deployed artifact's commit can't be found locally, before concluding it doesn't exist anywhere.
- ▶ Still deferred from doc 1: giving `docker-mcp`'s container an SSH client and key, and publishing the laptop's local Postgres to the network — both needed before cross-host targeting from the cloud gateway actually works end to end.

final-run - 3 of 3 · reconciled and pushed as 1e540a7